

فاز ۱۴: JDBC، JPA و Hibernate

۱۴.۱ JDBC (Java Database Connectivity)

چیست؟ پایه‌ای‌ترین راه ارتباط با دیتابیس در جاوا.

مراحل کار با JDBC:

```

1 // a. بارگذاری Driver
2 Class.forName("com.mysql.cj.jdbc.Driver");
3
4 // b. ایجاد Connection
5 Connection conn = DriverManager.getConnection(
6     "jdbc:mysql://localhost:3306/mydb",
7     "root",
8     "password"
9 );
10
11 // c. ساخت Statement
12 Statement stmt = conn.createStatement();
13
14 // d. اجرای Query
15 ResultSet rs = stmt.executeQuery("SELECT * FROM users");
16
17 // e. پردازش نتایج
18 while (rs.next()) {

```

```

19 String name = rs.getString("name");
20 int age = rs.getInt("age");
21 System.out.println(name + " - " + age);
22 }
23
24 // f. بستن منابع (مهم!)
25 rs.close();
26 stmt.close();
27 conn.close();

```

PreparedStatement و JDBC ۱۴.۱

PreparedStatement (مهم برای جلوگیری از SQL Injection) !

```

1 //  PreparedStatement
2 String query = "SELECT * FROM users WHERE username = ? AND password = ?";
3 PreparedStatement ps = conn.prepareStatement(query);
4 ps.setString(1, username);
5 ps.setString(2, password);
6 ResultSet rs = ps.executeQuery();
7
8 //  concatenation (خطر SQL Injection!)
9 String query = "SELECT * FROM users WHERE username = " + username + "'";
10 Statement stmt = conn.createStatement();
11 ResultSet rs = stmt.executeQuery(query);

```

JPA (Java Persistence API) ۱۴.۲

چیست؟ به استاندارد (Specification) برای Mapping اشیاء جاوا به جداول دیتابیس.

مفاهیم کلیدی:

مفهوم	توضیح
Entity@	کلاس رو به یه جدول دیتابیس تبدیل می‌کنه
Id@	کلید اصلی رو مشخص می‌کنه
GeneratedValue@	تولید خودکار مقدار کلید اصلی
Column@	نگاشت فیلد به ستون دیتابیس
OneToMany, @ManyToOne@	روابط بین جداول

```

1 @Entity
2 @Table(name = "users")
3 public class User {
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     @Column(nullable = false, unique = true)
9     private String username;
10
11    @Column(nullable = false)
12    private String password;
13
14    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)

```

```

15 private List<Order> orders;
16
17 // getters and setters
18 }

```

۱۴.۳ Hibernate (پیاده‌سازی JPA)

چیست؟ محبوب‌ترین پیاده‌سازی JPA که کارهای زیر رو انجام میده:

• تبدیل خودکار Object به SQL (و برعکس)

• مدیریت Cache (کش)

• مدیریت Transaction

• Lazy Loading (بارگذاری دیر هنگام)

! سوال مصاحبه‌ای: N+1 Query Problem چیه و چطور حل میشه؟

مشکل: وقتی یه Entity رو می‌خونی و بعد به Property که Lazy هست دسترسی پیدا می‌کنی، Hibernate یه Query جداگانه می‌زنه.

```

1 // a. این کوئری ۱ بار اجرا میشه
2 List<User> users = userRepository.findAll();
3
4 // b. (N بار) اینجا برای هر کاربر یه کوئری جداگانه زده میشه
5 for (User user : users) {
6     System.out.println(user.getOrders().size()); // N+1 Query!
7 }

```

• راه حل‌ها:

۱. JOIN FETCH

```
1 @Query("SELECT u FROM User u JOIN FETCH u.orders")
2 List<User> findAllWithOrders();
```

۲. EntityGraph

```
1 @EntityGraph(attributePaths = {"orders"})
2 List<User> findAll();
```

۳. @NamedEntityGraph

```
1 @Entity
2 @NamedEntityGraph(name = "User.orders", attributeNodes = @NamedAttributeNode("orders"))
3 public class User { ... }
```

! سوال مصاحبه‌ای: تفاوت FetchType.EAGER و LAZY

FetchType	توضیح	مزیت	عیب
EAGER	داده‌های مرتبط رو همین الان بارگذاری کن	ساده‌تر	کندتر، مصرف حافظه بیشتر
LAZY (پیش‌فرض)	فقط وقتی نیاز شد بارگذاری کن	سریع‌تر، کم مصرف	ممکنه به N+1 مشکل بخوری

۱۴.۴ مقایسه JDBC vs JPA vs Hibernate

1 (پایین ترین سطح) JDBC

2 |

3 ▼

4 JPA (Interface استاندارد - فقط)

5 |

6 ▼

7 Hibernate (پیاپیاده سازی - محبوب ترین)

8 |

9 ▼

10 Spring Data JPA (از Hibernate راحت تر)

ویژگی	JDBC	JPA	Hibernate
سطح	پایین	متوسط	بالا
کنترل	کامل	کمتر	کمترین
سرعت	سریع ترین	متوسط	کندتر (به خاطر Overhead)
راحتی	سخت	راحت	بسیار راحت
کد نویسی	زیاد	متوسط	کم

۱۴.۵ Spring Data JPA (لایه راحت تر از Hibernate)

Spring Data JPA به لایه بالاتر از Hibernate هست که نوشتن کدهای دیتابیس رو خیلی راحت تر می کنه.

۱۴.۵.۱ چرا Spring Data JPA؟

1 (خام Hibernate با) Spring Data JPA بدون a. //

2 Session session = entityManager.unwrap(Session.class);

3 Query query = session.createQuery("FROM User WHERE username = :username");

4 query.setParameter("username", "ali");

```

5 User user = (User) query.uniqueResult();
6
7 // b. خیلی ساده‌تر - Spring Data JPA
8 User user = userRepository.findByUsername("ali");

```

Spring Data در Repository ها ۱۴.۵.۲

```

1 // a. JpaRepository - کامل‌ترین
2 public interface UserRepository extends JpaRepository<User, Long> {
3     // متدهای آماده: save(), findById(), findAll(), delete(), count()
4 }
5
6 // b. CrudRepository - پایه‌ای‌تر
7 public interface UserRepository extends CrudRepository<User, Long> {}
8
9 // c. PagingAndSortingRepository - برای صفحه‌بندی
10 public interface UserRepository extends PagingAndSortingRepository<User, Long> {}

```

۱۴.۵.۳ متدهای Query Method (نوشتن کوئری با اسم متد)

```

1 public interface UserRepository extends JpaRepository<User, Long> {
2
3     // username پیدا کردن با
4     User findByUsername(String username);
5
6     // username و password پیدا کردن با
7     User findByUsernameAndPassword(String username, String password);

```

```
8
9  // پیدا کردن همه کاربران با سن بیشتر از
10 List<User> findByAgeGreaterThan(int age);
11
12  // پیدا کردن کاربرانی که اسمشون با ... شروع میشه
13 List<User> findByUsernameStartingWith(String prefix);
14
15  // مرتب سازی
16 List<User> findAllOrderByUsernameAsc();
17
18  // صفحه بندی
19 Page<User> findAll(Pageable pageable);
20
21  // بررسی وجود داشتن
22 boolean existsByUsername(String username);
23
24  // شمارش
25 long countByAgeGreaterThan(int age);
26
27  // حذف
28 void deleteByUsername(String username);
29 }
```

۱۴.۵.۴ Query@ (نوشتن کوئری دستی)

```

1 public interface UserRepository extends JpaRepository<User, Long> {
2
3     // ۱. JPQL (مخصوص JPA)
4     @Query("SELECT u FROM User u WHERE u.username = :username")
5     User findUserByUsername(@Param("username") String username);
6
7     // ۲. Native Query (SQL خام)
8     @Query(value = "SELECT * FROM users WHERE age > :age", nativeQuery = true)
9     List<User> findUsersOlderThan(@Param("age") int age);
10
11     // ۳. JOIN کوئری با
12     @Query("SELECT u FROM User u JOIN u.orders o WHERE o.total > :amount")
13     List<User> findUsersWithOrdersAbove(@Param("amount") double amount);
14 }

```

۱۴.۵.۵ Modifying@ (برای UPDATE و DELETE)

```

1 public interface UserRepository extends JpaRepository<User, Long> {
2
3     @Modifying
4     @Query("UPDATE User u SET u.active = false WHERE u.lastLogin < :date")
5     int deactivateInactiveUsers(@Param("date") LocalDate date);
6

```

```

7  @Modifying
8  @Query("DELETE FROM User u WHERE u.active = false")
9  int deleteInactiveUsers();
10 }

```

۱۴.۵.۶ صفحه بندی (Pagination)

```

1  @Service
2  class UserService {
3      @Autowired
4      private UserRepository repository;
5
6      public Page<User> getUsers(int page, int size) {
7          // صفحه ۰، هر صفحه ۱۰ تا
8          Pageable pageable = PageRequest.of(page, size);
9          return repository.findAll(pageable);
10     }
11
12     public Page<User> getSortedUsers(int page, int size) {
13         Pageable pageable = PageRequest.of(page, size, Sort.by("username").ascending());
14         return repository.findAll(pageable);
15     }
16 }

```

۱۴.۵.۷ Specification (کوئری داینامیک)

```

1  // a. برای وقتی که فیلترها متغیر هستند
2  public class UserSpecification {

```

```

3  public static Specification<User> hasUsername(String username) {
4      return (root, query, criteriaBuilder) ->
5          criteriaBuilder.equal(root.get("username"), username);
6  }
7
8  public static Specification<User> isActive() {
9      return (root, query, criteriaBuilder) ->
10         criteriaBuilder.isTrue(root.get("active"));
11  }
12 }
13
14 // b. استفاده
15 @Service
16 class UserService {
17     public List<User> filterUsers(String username, Boolean active) {
18         Specification<User> spec = Specification.where(null);
19
20         if (username != null) {
21             spec = spec.and(UserSpecification.hasUsername(username));
22         }
23         if (active != null) {
24             spec = spec.and(UserSpecification.isActive());
25         }
27         return repository.findAll(spec);
28     }
29 }

```